

Progetto Intelligenza Artificiale

2022/2023

Ilenia Galati 242600

Carmen D'Agostino 247471

Maria Critelli 247340

Indice

1	Modeling	2
1.1	Il dominio	2
1.1.1	I tipi	2
1.1.2	I predicati	3
1.1.3	Le azioni	5
2	Classical Planning	12
2.1	Planner	12
2.2	Risultati	25
2.2.1	Istanza 1	26
2.2.2	Istanza 2	29
2.2.3	Istanza 3	35
3	Temporal Planning & Robotics	47
3.1	Temporal Planning	47
3.1.1	Modifiche al file di dominio	47
3.1.2	Modifiche al file di problema	54
3.1.3	Ottenimento del piano	55
3.2	Robotics Planning	57
3.2.1	File del workspace	58
3.2.2	Simulazione	74

1 Modeling

Il progetto ha come obiettivo la modellazione di un problema ispirato alla logistica dei servizi di emergenza. In questo scenario, numerose persone ferite si trovano in posizioni note e fisse. Il compito degli agenti robotici è quello di consegnare scatole contenenti forniture di emergenza a ciascuna persona. La modellazione considera diversi aspetti come la gestione del contenuto delle scatole, la capacità di trasporto dei robot e la tracciabilità della distribuzione delle risorse.

1.1 Il dominio

Questa prima fase di modellazione del problema di planning comprende la definizione del file di dominio attraverso la versione 1.2 del linguaggio PDDL, nel quale vengono definiti i diversi tipi di oggetti, predicati e requisiti dello scenario.

Sono stati impiegati, per la modellazione i seguenti requirements di PDDL:

```
1 (define
2   (domain emergency)
3   (:requirements :strips :typing))
```

1.1.1 I tipi

Per quanto riguarda la definizione dei types, si è deciso di introdurre "locatable" per riuscire a modellare gli oggetti che è importante localizzare.

```
1   (:types
2     location - object
3     carrierplace - object
4     locatable - object
5     box - locatable
6     agent - locatable
7     content - locatable)
```

```

8         carrier - locatable
9         person - locatable
10    )

```

- **location:** luoghi in cui possono trovarsi persone o altri oggetti del dominio;
- **carrierplace:** serve per tenere sotto controllo i posti disponibili sui carriers;
- **box:** cassette da riempire con gli oggetti da trasportare;
- **agent:** robot che eseguono le azioni al fine di consegnare i diversi contents;
- **content:** il contenuto inseribile nelle box (strumenti, medicine e cibo);
- **carrier:** vettori su cui i robot caricano le boxes;
- **person:** persone a cui consegnare i contents.

1.1.2 I predicati

Si procede con la definizione di proprietà e relazioni dei diversi oggetti attraverso i seguenti predicati:

```

1  (:predicates
2      (empty ?b - box)
3      (carrierPlace ?p - carrierplace ?c - carrier)
4      (availablePlace ?p - carrierplace)
5      (boxOnPlace ?b - box ?p - carrierplace)
6      (at ?o - locatable ?l - location)
7      (has ?p - person ?c - content)
8      (need ?p - person ?c - content)
9      (inBox ?b - box ?c - content)
10     (hasSomething ?p - person)
11     (needAll ?p - person)

```

```

12         (needSomething ?p - person)
13     )

```

- **(empty ?b - box)** Per definire se una scatola è vuota;
- **(carrierPlace ?p - carrierplace ?c - carrier)** Per definire se il posto appartiene al vettore;
- **(availablePlace ?p - carrierplace)** Per definire se il posto è libero;
- **(boxOnPlace ?b - box ?p - carrierplace)** Per definire se la box si trova sul carrier;
- **(at ?o - locatable ?l - location)** Per definire se l'oggetto localizzabile si trova nella location;
- **(has ?p - person ?c - content)** Per definire se la persona possiede il content;
- **(need ?p - person ?c - content)** Per definire se la persona ha bisogno del content;
- **(inBox ?b - box ?c - content)** Per definire se la box contiene il content;
- **(hasSomething ?p - person)** Per definire se la persona possiede almeno una risorsa tra quelle di cui ha bisogno;
- **(needAll ?p - person)** Per definire se la persona ha bisogno che le vengano recapitate tutte le risorse per ritenersi soddisfatta;
- **(needSomething ?p - person)** Per definire se la persona ha bisogno di almeno una risorsa recapitata tra quelle di cui ha bisogno per ritenersi soddisfatta.

1.1.3 Le azioni

Sono state definite in tutto 6 azioni nel file di dominio, elencate di seguito.

Fill Necessaria per riempire una box con un content, l'azione *fill* necessita come parametri di un *agent*, una *box*, un *content* e una *location*. Sono necessarie, come precondizioni:

- la box deve essere vuota;
- l'agent si trovi nella location specificata;
- la box si trovi nella location specificata;
- il content si trovi nella location specificata.

Gli effetti dell'azione:

- inserimento del content nella box;
- la box non sarà più vuota.

```
1 (:action fill
2       :parameters (?a - agent ?b - box ?c - content ?
3       l - location)
4       :precondition (and
5           (empty ?b)
6           (at ?a ?l)
7           (at ?b ?l)
8           (at ?c ?l)
9       )
10      :effect (and
11          (not(empty ?b))
12          (inBox ?b ?c)
13      )
14  )
```

Move Necessaria per lo spostamento di un robot da una location ad un'altra, l'azione *move* necessita come parametri di un *agent*, un *carrier*, una *location* di partenza e una di arrivo. Sono necessarie, come precondizioni:

- l'agent si trovi nella location di partenza;
- la box si trovi nella location di partenza.

Gli effetti di questa azione consistono nello spostamento di carrier e agent. In particolare:

- il carrier non si trova nella posizione di partenza e si trova in quella di arrivo;
- l'agent non si trova nella posizione di partenza e si trova in quella di arrivo.

```
1 (:action move
2   :parameters (?a - agent ?c - carrier ?from ?to -
3     location)
4   :precondition (and
5     (at ?a ?from)
6     (at ?c ?from))
7   :effect (and
8     (at ?a ?to)
9     (at ?c ?to)
10    (not (at ?a ?from))
11    (not (at ?c ?from))
12  )
13 )
```

Empty L'azione è necessaria per svuotare la box al momento della consegna delle risorse nel caso di persone che richiedono la consegna di tutte le risorse di cui necessitano. L'azione Empty coinvolge un *agent*, una *box*, un *content*, una *person* e una *location*. Sono necessarie, come precondizioni:

- il content si trovi all'interno della box;
- l'agent si trovi nella location;
- la persona si trovi nella location;
- la box si trovi nella location;
- la persona ha bisogno del content;
- la persona si ritiene soddisfatta solo quando avrà ricevuto tutti i contents di cui ha bisogno.

L'azione Empty ha come effetti:

- il content non si trova più nella box;
- la box è vuota;
- la persona non necessita più del content;
- la persona possiede il content.

```

1 (:action empty
2       :parameters (?a - agent ?b - box ?c - content ?
3       p - person ?l - location)
4       :precondition (and
5           (inBox ?b ?c)
6           (at ?a ?l)
7           (at ?b ?l)
8           (at ?p ?l)
9           (need ?p ?c)
10          (needAll ?p) )
11       :effect (and
12           (not (inBox ?b ?c))
13           (empty ?b)
14           (not (need ?p ?c))
15           (has ?p ?c))
16   )

```

EmptyAtLeastOne L'azione è necessaria per svuotare la box al momento della consegna delle risorse nel caso di persone che richiedono la consegna di almeno una tra tutte le risorse che necessitano. L'azione coinvolge un *agent*, una *box*, un *content*, una *person* e una *location*. Sono necessarie, come precondizioni:

- il content si trovi all'interno della box;
- l'agent si trovi nella location;
- la persona si trovi nella location;
- la box si trovi nella location;
- la persona ha bisogno del content;
- la persona si ritiene soddisfatta solo quando avrà ricevuto almeno uno fra i contents di cui ha bisogno.

L'azione ha come effetti:

- il content non si trova più nella box;
- la box è vuota;
- la persona non necessita più di alcun content;
- la persona possiede almeno un content fra quelli di cui necessitava.

```
1 (:action emptyAtLeastOne
2       :parameters (?a - agent ?b - box ?c - content ?
3       p - person ?l - location)
4       :precondition (and
5           (inBox ?b ?c)
6           (at ?a ?l)
7           (at ?b ?l)
8           (at ?p ?l)
9           (need ?p ?c)
           (needSomething ?p))
```

```

10         :effect (and
11             (not (inBox ?b ?c))
12             (empty ?b)
13             (not (need ?p ?c))
14             (not (needSomething ?p))
15             (hasSomething ?p) )
16     )

```

Load Necessaria per caricare la box sul carrier per essere trasportata in un'altra location, l'azione necessita come parametri di un *agent*, un *carrier*, un *carrierplace*, una *box* e una *location*. Sono necessarie, come precondizioni:

- l'agent si trovi nella location;
- il carrier si trovi nella location;
- la box si trovi nella location;
- il posto su cui caricare la box appartenga a quel carrier;
- il posto su cui caricare la box deve essere libero.

L'azione ha come effetti:

- il posto del carrier è occupato dalla box;
- la box non si trova più nella location;
- il posto non è più libero.

```

1 (:action load
2   :parameters (?a - agent ?c - carrier ?p -
3     carrierplace ?b - box ?l - location)
4   :precondition (and
5     (at ?a ?l)
6     (at ?c ?l)
7     (at ?b ?l)
8     (carrierPlace ?p ?c)

```

```

8             (availablePlace ?p)
9         )
10        :effect (and
11            (boxOnPlace ?b ?p)
12            (not(at ?b ?l))
13            (not(availablePlace ?p))
14        )
15    )

```

Unload Necessaria per scaricare la box dal carrier, l'azione necessita come parametri di un *agent*, un *carrier*, un *carrierplace*, una *box* e una *location*. Sono necessarie, come precondizioni:

- l'agent si trovi nella location;
- il carrier si trovi nella location;
- la box si trovi sul posto specificato;
- il posto da cui fare unload sia un posto del carrier specificato.

L'azione ha come effetti:

- la box non si trova più in quel posto;
- la box non si trova più nella location;
- il posto è adesso libero.

```

1 (:action unload ;scaricare la box dal carrier
2     :parameters (?a - agent ?c - carrier ?p -
3         carrierplace ?b - box ?l - location)
4     :precondition (and
5         (at ?a ?l)
6         (at ?c ?l)
7         (boxOnPlace ?b ?p)
8         (carrierPlace ?p ?c)
9     )
10     :effect (and

```

```
10          (not (boxOnPlace ?b ?p))
11          (at ?b ?l)
12          (availablePlace ?p)
13      )
14  )
```

2 Classical Planning

Per la realizzazione del planner custom e la risoluzione delle tre istanze di un problema di pianificazione si è fatto uso di PDDL4J, una libreria Java progettata per supportare lo sviluppo di planner, dotata di una serie di strumenti e funzionalità per leggere, manipolare e risolvere problemi descritti in PDDL.

2.1 Planner

Il planner realizzato utilizza un'euristica per guidare la ricerca ed implementa un'opzione di pruning stocastico per migliorarne l'efficienza. La classe ASP estende la classe *AbstractPlanner* ed apporta alcune modifiche all'algoritmo A*, consentendo agli utenti di selezionare diverse euristiche e di configurare il peso di queste. Una prima differenza rispetto all'impiego della classica funzione $f(x)$ in A* è infatti l'introduzione di un fattore moltiplicativo (w) che definisce il peso dell'euristica:

$$f(x) = w \cdot h(x) + g(x)$$

Tale contributo permette di adattare il planner a diversi tipi di problemi di pianificazione, ottimizzando l'efficienza della ricerca e migliorando le prestazioni complessive. Per lo stesso motivo è stato utilizzato il pruning stocastico, implementato nella funzione `astarSearch`, con una probabilità del 50 di generare lo stato figlio se il pruning è attivo. Il codice permette inoltre di configurare diversi tipi di euristiche tramite opzioni di comando.

Di seguito viene riportata la classe ASP:

```
1 public class ASP extends AbstractPlanner {  
2     private static final Logger LOGGER = LogManager.  
    getLogger(ASP.class.getName());  
3     public static final String HEURISTIC_SETTING = "  
    HEURISTIC";  
4     public static final StateHeuristic.Name  
    DEFAULT_HEURISTIC = StateHeuristic.Name.AJUSTED_SUM;
```

```

5      /**
6       * The WEIGHT_HEURISTIC property used for planner
configuration.
7       */
8      public static final String WEIGHT_HEURISTIC_SETTING = "
WEIGHT_HEURISTIC";
9      /**
10     * The default value of the WEIGHT_HEURISTIC property
used for planner configuration.
11     */
12     public static final double DEFAULT_WEIGHT_HEURISTIC =
1.0;
13     /**
14     * The stochastic pruning property for deciding if a
node must be dropped.
15     */
16     public static boolean stochasticPruning = false;
17
18     /**
19     * The total number of states evaluated
20     */
21     private int numOfStatesEvaluated = 0;
22     /**
23     * The weight of the heuristic.
24     */
25     private double heuristicWeight;
26     /**
27     * The name of the heuristic used by the planner.
28     */
29     private StateHeuristic.Name heuristic;
30     /**
31     * Creates a new A* search planner with a specified
configuration.
32     * @param configuration the configuration of the planner
33     */
34     public ASP(final PlannerConfiguration configuration) {
35         super();

```

```

36         this.setConfiguration(configuration);
37     }
38
39 }

```

Alcuni dei parametri configurabili includono:

- **Heuristic Weight** (`-weight` o `-w`): peso dell'euristica utilizzata nella funzione di valutazione.
- **Stochastic Pruning** (`-stochastic-pruning` o `-sp`): flag per abilitare/disabilitare il pruning stocastico, che può ridurre lo spazio di ricerca esplorato.
- **Heuristic** (`-heuristic` o `-e`): nome dell'euristica da utilizzare (es. FAST FORWARD, AJUSTED SUM). L'euristica di default è AJUSTED SUM.

```

1  /**
2      * Sets the weight of the heuristic.
3      * @param weight the weight of the heuristic. The weight
4      * must be greater than 0.
5      * @throws IllegalArgumentException if the weight is
6      * strictly less than 0.
7      */
8      @CommandLine.Option(names = {"-w", "--weight"},
9      defaultValue = "1.0",
10      paramLabel = "<weight>", description = "Set the
11      weight of the heuristic (preset 1.0).")
12      public void setHeuristicWeight(final double weight) {
13          if (weight <= 0) {
14              throw new IllegalArgumentException("Weight <= 0"
15          );
16          }
17          this.heuristicWeight = weight;
18      }
19  /**
20      * Set the stochastic pruning flag.

```

```

16     * @param value the assigned value to the flag.
17     */
18     @CommandLine.Option(names = {"-sp", "--stochastic-
19     pruning"}, defaultValue = "false",
20         paramLabel = "<value>", description = "Set the
21     stochastic-pruning flag (preset false).")
22     public void setStochasticPruning(final String value) {
23
24         this.stochasticPruning = value.equalsIgnoreCase("
25     true") ? true : false;
26     }
27
28     /**
29     * Set the name of heuristic used by the planner to the
30     solve a planning problem.
31     *
32     * @param heuristic the name of the heuristic.
33     */
34     @CommandLine.Option(names = {"-e", "--heuristic"},
35     defaultValue = "AJUSTED_SUM",
36         description = "Set the heuristic : FAST_FORWARD,
37     AJUSTED_SUM, AJUSTED_SUM2, AJUSTED_SUM2M, COMBO, "
38     + "MAX, FAST_FORWARD SET_LEVEL, SUM,
39     SUM_MUTEX (preset: FAST_FORWARD)")
40     public void setHeuristic(StateHeuristic.Name heuristic)
41     {
42         this.heuristic = heuristic;
43     }

```

Successivamente si mostrano i metodi per istanziare un problema di pianificazione, verificare che la configurazione del planner sia valida e determinare la configurazione del planner con i parametri impostati; se il parametro non viene specificato nella configurazione viene inizializzato con il valore default precedentemente definito.

```

1     /**
2     * Instantiates the planning problem from a parsed
3     problem.
4     *
5     * @param problem the problem to instantiate.

```

```

4      * @return the instantiated planning problem or null if
the problem cannot be instantiated.
5      */
6      @Override
7      public Problem instantiate(DefaultParsedProblem problem)
      {
8          final Problem pb = new DefaultProblem(problem);
9          pb.instantiate();
10         return pb;
11     }
12
13     /**
14      * Checks the planner configuration and returns if the
configuration is valid.
15      * A configuration is valid if (1) the domain and the
problem files exist and
16      * can be read, (2) the timeout is greater than 0, (3)
the weight of the
17      * heuristic is greater than 0 and (4) the heuristic is
a not null.
18      * @return <code>true</code> if the configuration is
valid <code>false</code> otherwise.
19      */
20     public boolean isValidConfiguration() {
21         return super.isValidConfiguration()
22             && this.getHeuristicWeight() > 0.0
23             && this.getHeuristic() != null;
24     }
25     /**
26      * This method return the default arguments of the
planner.
27      * @return the default arguments of the planner.
28      * @see PlannerConfiguration
29      */
30     public static PlannerConfiguration
getDefaultConfiguration() {
31         PlannerConfiguration config = Planner.

```

```

getDefaultConfiguration();
32     config.setProperty(ASP.HEURISTIC_SETTING, ASP.
DEFAULT_HEURISTIC.toString());
33     config.setProperty(ASP.WEIGHT_HEURISTIC_SETTING,
34         Double.toString(ASP.DEFAULT_WEIGHT_HEURISTIC
));
35     return config;
36 }
37 /**
38  * Returns the configuration of the planner.
39  * @return the configuration of the planner.
40  */
41 @Override
42 public PlannerConfiguration getConfiguration() {
43     final PlannerConfiguration config = super.
getConfiguration();
44     config.setProperty(ASP.HEURISTIC_SETTING, this.
getHeuristic().toString());
45     config.setProperty(ASP.WEIGHT_HEURISTIC_SETTING,
Double.toString(this.getHeuristicWeight()));
46     return config;
47 }
48
49 /**
50  * Sets the configuration of the planner. If a planner
setting is not defined in
51  * the specified configuration, the setting is
initialized with its default value.
52  * @param configuration the configuration to set.
53  */
54 @Override
55 public void setConfiguration(final PlannerConfiguration
configuration) {
56     super.setConfiguration(configuration);
57     if (configuration.getProperty(ASP.
WEIGHT_HEURISTIC_SETTING) == null) {
58         this.setHeuristicWeight(ASP.

```

```

        DEFAULT_WEIGHT_HEURISTIC);
59     } else {
60         this.setHeuristicWeight(Double.parseDouble(
        configuration.getProperty(
61             ASP.WEIGHT_HEURISTIC_SETTING)));
62     }
63     if (configuration.getProperty(ASP.HEURISTIC_SETTING)
        == null) {
64         this.setHeuristic(ASP.DEFAULT_HEURISTIC);
65     } else {
66         this.setHeuristic(StateHeuristic.Name.valueOf(
        configuration.getProperty(
67             ASP.HEURISTIC_SETTING)));
68     }
69 }

```

Infine, si mostra il metodo per la ricerca di un piano di soluzione per le istanze specificate utilizzando l'algoritmo A*.

```

1  /**
2   * Search a solution plan to a specified domain and
   * problem using A*.
3   * @param problem the problem to solve.
4   * @return the plan found or null if no plan was found.
5   */
6  @Override
7  public Plan solve(final Problem problem) {
8      LOGGER.info("Starting search..");
9
10     long startTime = System.currentTimeMillis();
11     Plan plan = astarSearch(problem);
12     long endTime = System.currentTimeMillis();
13
14     // If a plan is found update the statistics of the
   * planner and log search information
15     if (plan != null) {
16         LOGGER.info("* A* search succeeded\n");
17         LOGGER.info("Number of states evaluated : " +

```

```

        this.numOfStatesEvaluated);
18         this.getStatistics().setTimeToSearch(endTime -
            startTime);
19     } else {
20         LOGGER.info("* A* search failed\n");
21     }
22
23     // Return the plan found or null if the search fails
24     .
25     return plan;
}

```

Inizia la ricerca registrando il tempo di inizio ed utilizzando il metodo `astarSearch` illustrato in seguito. Se viene trovato un piano, si prosegue aggiornando le statistiche del planner e registrando le informazioni della ricerca. Se la ricerca ha successo, vengono mostrati il numero di stati valutati e il tempo impiegato. Se la ricerca fallisce, viene registrato il fallimento. Il metodo restituisce il piano trovato o null qual ora non fosse stato determinato alcun piano.

Il metodo **`astarSearch`** implementa l'algoritmo A* per la risoluzione dei problemi di pianificazione, sfruttando una combinazione di coda di priorità, euristica e una gestione efficiente dei nodi esplorati attraverso l'utilizzo di un `HashMap`. Tale struttura tiene traccia sia degli stati completamente esplorati durante la ricerca, sia degli stati ancora in fase di esplorazione. Ogni stato esplorato è memorizzato come chiave con il corrispondente nodo (oggetto `Node`) come valore associato. Questo aiuta a garantire che uno stato già completamente esplorato non venga esplorato nuovamente, ottimizzando così le risorse computazionali.

```

1 /**
2     * Search for a plan using A* algorithm.
3     * @param problem the problem for which the plan must be
        found.
4     * @return a plan if it is found within the time limit,
        otherwise null.

```

```

5      */
6
7      public Plan astarSearch(Problem problem) {
8          StateHeuristic heuristic = StateHeuristic.
getInstance(this.getHeuristic(), problem);
9          Random random = new Random();
10         // Get the initial state from the planning problem
11         State init = new State(problem.getInitialState());
12         // An HashMap was used instead of a set in order to
have a data structure that has constant access time and
membership checking
13         // With a set the former would be O(N) instead of O
(1)
14         Map<State, Node> exploredNodes = new HashMap<>();
15         Map<State, Node> nodesToExploreSet = new HashMap<>()
;
16         double heuristicWeight = this.getHeuristicWeight();
17         // The list stores the node ordered according to the
A* (getFValue = g + h) function
18         PriorityQueue<Node> nodesToExplore = new
PriorityQueue<Node>(problem.getActions().size(), new
Comparator<Node>() {
19             public int compare(Node n1, Node n2) {
20                 double f1 = heuristicWeight * n1.
getHeuristic() + n1.getCost();
21                 double f2 = heuristicWeight * n2.
getHeuristic() + n2.getCost();
22                 return Double.compare(f1, f2);
23             }
24         });

```

La frontiera viene dunque inizializzata utilizzando una coda di priorità basata sulla funzione di valutazione $f(x) = w \cdot h(x) + g(x)$, dove

- w rappresenta il peso dell'euristica
- h è il valore euristico del nodo, una stima del costo rimanente per raggiungere l'obiettivo finale dallo stato n

- g è il costo accumulato finora per raggiungere lo stato n

Il comparatore personalizzato determina l'ordine di estrazione degli oggetti Node dalla coda di priorità. La classe Node estende la classe State di PDDL4J e rappresenta un nodo dell'albero di ricerca generato. Tale classe contiene le informazioni necessarie alla ricerca come il costo del nodo raggiunto, la profondità e l'azione che l'ha generato.

I valori f_1 e f_2 per i nodi n_1 e n_2 sono calcolati considerando il peso dell'euristica (weight), l'euristica del nodo ($n_1.getHeuristic()$), e il costo accumulato fino a quel punto ($n_1.getCost()$). La stessa procedura è applicata a n_2 . Infine, il comparatore utilizza 'Double.compare' per confrontare f_1 e f_2 , stabilendo così l'ordine di estrazione dalla coda di priorità.

Il cuore dell'algoritmo è il ciclo principale di ricerca, che continua finché ci sono nodi da esplorare nella coda di priorità. Ad ogni iterazione, viene estratto il nodo con il costo stimato minore $f(n)$ e valutato per determinare se rappresenta una soluzione al problema.

```

1 // Creates the root node of the tree search
2     Node root = new Node(init, null, -1, 0, heuristic.
      estimate(init, problem.getGoal()));
3     // Adds the root to the list of nodes to be explored
4     nodesToExplore.add(root);
5     nodesToExploreSet.put(init, root);
6
7     // even though the termination condition is for the
      search space to completely explored,
8     // the algorithm terminates much earlier
9     while (!nodesToExplore.isEmpty()) {
10         // Pop the first node in the pending list open
11         Node current = nodesToExplore.poll();
12         // updating the number of states visited
13         this.numOfStatesEvaluated += 1;
14         // removing the node from the set of explored
      nodes

```

```

15         nodesToExploreSet.remove(current);
16         exploredNodes.put(current, current);
17         // If the goal is reached then extract a plan
and return it
18         if (current.satisfy(problem.getGoal())) return
extractPlan(current, problem);
19         // Try to apply the operators of the problem to
this node
20         int i = 0;
21
22         for (Action op : problem.getActions()) {
23
24             // Test if a specified operator is
applicable in the current state (i.e verifying whether
the precondition
25             // of the action are satisfied)
26             if (op.isApplicable(current)) {
27                 Node state = new Node(current);
28                 // based on the value of the flag
stochastic pruning, we must distinguish between two
cases; the
29                 // first is whether it's value is false;
in this case we proceed to generate ALL the children
state.
30                 // In the remaining case, that is when
stochastic pruning is enabled, there is a probability of
50%
31                 // of generating the child state of the
node current.
32                 boolean generateChild = random.
nextBoolean();
33                 if ((this.stochasticPruning &&
generateChild) || (!this.stochasticPruning)) {
34                     List<ConditionalEffect> effects = op
.getConditionalEffects();
35                     // Apply the effect to the child
node (state)

```

```

36         applyEffects(effects, current, state
37     );
38
39     double g = current.getCost() + op.
40     getCost().getValue();
41     Node result = nodesToExploreSet.get(
42     state);
43
44     // if the child node is yet to be
45     explored
46     if (result == null) {
47         result = exploredNodes.get(state
48     );
49
50     // if the child node was already
51     explored precedently
52     if (result != null) {
53         // if the cumulative cost of
54         the node currently generated is lesser than the
55         // node precedently explored
56         , then we update it's fields and reinsert into the
57         // list of nodes to be
58         explored
59
60         if (g < result.getCost()) {
61             updateNode(result,
62             current, g, i);
63
64             nodesToExplore.add(
65             result);
66
67             nodesToExploreSet.put(
68             result, result);
69
70             exploredNodes.remove(
71             result);
72
73         }
74
75         // otherwise we have a node
76         that was not explored and yet to be inserted in
77         // the list of nodes to
78         explore
79
80     } else {

```

```

58         updateNode(state, current, g
, i);
59         state.setHeuristic(heuristic
.estimate(state, problem.getGoal()));
60         nodesToExplore.add(state);
61         nodesToExploreSet.put(state,
state);
62     }
63     // otherwise, if it was already
added to the queue, we must check if it's cumulative
cost
64     // is lesser than the one
currently generated. In the positive case that it is, we
update the
65     // node in the list of the
pending nodes with the lesser cost.
66     } else if (g < result.getCost()) {
67         updateNode(result, current, g, i
);
68     }
69 }
70 }
71 i++;
72 }
73 }
74 // return the search computed or null if no search
was found
75 return null;
76 }

```

Durante ciascuna iterazione del ciclo:

- viene estratto il nodo con il valore di $f(n)$ minimo dalla coda di priorità;
- il nodo estratto viene esplorato, aggiornando il conteggio degli stati valutati;

- se il nodo corrente soddisfa il goal del problema, viene estratto e restituito un piano di soluzione tramite la funzione `extractPlan`.

Per ogni azione, viene generato uno stato successivo (state) applicando gli effetti dell'azione corrente e aggiornando il costo accumulato `g` con il costo di questa. È stato inoltre effettuato un controllo per gestire la generazione dei figli in base alla modalità di pruning stocastico (`stochasticPruning`). Il metodo `random.nextBoolean()` restituisce un valore booleano casuale (`true` o `false`); se `stochasticPruning` è abilitato (`true`), allora la generazione dei figli avviene solo se `generateChild` è `true`. In altre parole, c'è una probabilità del 50 che un nodo figlio venga effettivamente generato per ciascuna azione applicabile. Il pruning stocastico può accelerare il processo di ricerca saltando alcuni nodi potenzialmente non promettenti.

La possibilità di configurare dinamicamente l'euristica, il peso dell'euristica e il pruning stocastico consente di adattare l'algoritmo A^* alle specifiche necessità di ciascun problema. Questa flessibilità consente di adattare il comportamento dell'algoritmo alle caratteristiche specifiche di ciascun problema, ottimizzando così l'efficienza della ricerca.

2.2 Risultati

Di seguito vengono mostrati i risultati ottenuti per le 3 istanze, utilizzando il planner e l'euristica definiti in precedenza. In particolare, per l'esecuzione delle tre istanze è stata impiegata `ajusted sum` per guidare la selezione dei nodi da espandere durante la ricerca del percorso ottimale. Per ciascuna delle tre istanze è stato impostato un valore appropriato per il peso dell'euristica. Questo peso può essere regolato in base alla complessità del problema specifico e alla preferenza per l'equilibrio tra costo passato e stima del costo futuro.

Inoltre, è stata implementata la possibilità di configurare il comportamento dell'algoritmo A^* tramite l'abilitazione di "stochastic pruning"

direttamente da linea di comando. Questa tecnica consente di ottimizzare l'esplorazione dello spazio di ricerca, limitando la generazione di nodi figli in modo casuale per ridurre la complessità computazionale dell'algoritmo senza compromettere la qualità della soluzione.

2.2.1 Istanza 1

È stato definito inizialmente il primo problem file, riportato di seguito.

```
1 (define
2   (problem problem1)
3   (:domain emergency)
4
5   (:objects
6     a - agent
7     place1 place2 place3 place4 - carrierplace
8     depot l1 l2 - location
9     c - carrier
10    p1 p2 p3 - person
11    b1 b2 b3 b4 b5 - box
12    food medicine tools - content
13  )
14
15  (:init
16    (empty b1)
17    (empty b2)
18    (empty b3)
19    (empty b4)
20    (empty b5)
21
22    (at a depot)
23    (at c depot)
24    (at b1 depot)
25    (at b2 depot)
26    (at b3 depot)
27    (at b4 depot)
```

```

28      (at b5 depot)
29      (at tools depot)
30      (at medicine depot)
31      (at food depot)
32      (at p1 l1)
33      (at p2 l1)
34      (at p3 l2)
35
36      (availablePlace place1)
37      (availablePlace place2)
38      (availablePlace place3)
39      (availablePlace place4)
40
41      (carrierPlace place1 c)
42      (carrierPlace place2 c)
43      (carrierPlace place3 c)
44      (carrierPlace place4 c)
45
46      (need p1 food)
47      (need p1 medicine)
48      (need p2 medicine)
49      (need p3 food)
50
51      (needAll p1)
52      (needAll p2)
53      (needAll p3)
54  )
55
56  (:goal (and
57    (has p1 food)
58    (has p1 medicine)
59    (has p2 medicine)
60    (has p3 food))
61  )
62 )

```

Condizioni iniziali:

- cinque scatole si trovano nel deposito;
- tutti i contenuti necessari per caricare le scatole sono presenti nel deposito;
- tre persone: p1, p2 e p3. p1 e p2 sono nello stesso luogo. p1 necessita di cibo e medicine, p2 di medicine e p3 di cibo;
- un singolo agente robotico e un trasportatore si trovano nel deposito;
- ogni trasportatore può caricare fino a 4 scatole.

Obiettivo: Tutte le persone ferite devono ricevere gli oggetti di cui hanno bisogno.

Per la prima istanza è stato utilizzato un peso pari a 5.

```

1 Starting search... A* search succeeded
2 Number of states evaluated : 1645
3 found plan as follows:
4
5 00: ( fill a b5 medicine depot) [0]
6 01: ( load a c place1 b5 depot) [0]
7 02: (      move a c depot l1) [0]
8 03: (  unload a c place1 b5 l1) [0]
9 04: (      move a c l1 depot) [0]
10 05: (    fill a b1 food depot) [0]
11 06: ( load a c place1 b1 depot) [0]
12 07: (      move a c depot l1) [0]
13 08: (  unload a c place1 b1 l1) [0]
14 09: (    empty a b1 food p1 l1) [0]
15 10: (      move a c l1 depot) [0]
16 11: ( fill a b4 medicine depot) [0]
17 12: ( load a c place3 b4 depot) [0]
18 13: (      move a c depot l1) [0]
19 14: (empty a b5 medicine p1 l1) [0]
20 15: (  unload a c place3 b4 l1) [0]
21 16: (empty a b4 medicine p2 l1) [0]

```

```

22 17: (      move a c l1 depot) [0]
23 18: (      fill a b2 food depot) [0]
24 19: ( load a c place1 b2 depot) [0]
25 20: (      move a c depot 12) [0]
26 21: ( unload a c place1 b2 12) [0]
27 22: (      empty a b2 food p3 12) [0]
28
29 time spent:      0,01 seconds parsing
30                  0,03 seconds encoding
31                  0,28 seconds searching
32                  0,32 seconds total time
33
34 memory used:      0,68 MBytes for problem representation
35                  0,00 MBytes for searching
36                  0,68 MBytes total

```

Utilizzando il pruning stocastico, abilitato settando a true la relativa flag (-sp true) è possibile notare una riduzione del tempo impiegato.

```

1 time spent:      0,02 seconds parsing
2                  0,03 seconds encoding
3                  0,23 seconds searching
4                  0,28 seconds total time
5
6 memory used:      0,68 MBytes for problem representation
7                  0,00 MBytes for searching
8                  0,68 MBytes total

```

2.2.2 Istanza 2

Per la definizione della seconda istanza di problema, è stato definito il seguente file.

```

1 (define
2 (problem problem2)
3 (:domain emergency)
4
5 (:objects
6   a1 a2 - agent

```

```

7      c1 c2 - carrier
8      depot l1 l2 l3 l4 l5 - location
9      food medicine tools - content
10     place1 place2 place3 place4 - carrierplace
11     b1 b2 b3 - box
12     p1 p2 p3 p4 p5 p6 - person
13 )
14
15 (:init
16     (empty b1)
17     (empty b2)
18     (empty b3)
19
20     (availablePlace place1)
21     (availablePlace place2)
22     (availablePlace place3)
23     (availablePlace place4)
24
25     (at a1 depot)
26     (at a2 depot)
27     (at c1 depot)
28     (at c2 depot)
29     (at b1 depot)
30     (at b2 depot)
31     (at b3 depot)
32     (at food depot)
33     (at medicine depot)
34     (at tools depot)
35     (at p1 l1) ;ci serve garantire che p1 e p2 siano
nello stesso luogo e che gli altri siano in luoghi
diversi
36     (at p2 l1)
37     (at p3 l2)
38     (at p4 l3)
39     (at p5 l4)
40     (at p6 l5)
41

```

```

42      (carrierPlace place1 c1)
43      (carrierPlace place2 c1)
44      (carrierPlace place3 c2)
45      (carrierPlace place4 c2)
46
47      (need p1 food) ;p1 needs food or tools
48      (need p1 tools)
49      (need p2 medicine) ;p2 needs medicine
50      (need p3 medicine) ;p3 needs medicine
51      (need p4 medicine) ;p4 need medicine and food
52      (need p4 food)
53      (need p5 medicine) ;p5 and p6 needs everything.
54      (need p5 food)
55      (need p5 tools)
56      (need p6 medicine)
57      (need p6 food)
58      (need p6 tools)
59
60      (needSomething p1) ;ci basta che abbia uno tra food e
tools
61      (needAll p2)
62      (needAll p3)
63      (needAll p4)
64      (needAll p5)
65      (needAll p6)
66 )
67
68 (:goal (and
69      (hasSomething p1)
70      (has p2 medicine)
71
72      (has p3 medicine)
73
74      (has p4 medicine)
75      (has p4 food)
76
77      (has p5 food)

```

```

78     (has p5 medicine)
79     (has p5 tools)
80
81     (has p6 food)
82     (has p6 medicine)
83     (has p6 tools))
84 )
85
86 )

```

Condizioni iniziali:

- due robot e due trasportatori;
- tre scatole;
- capacità del trasportatore: 2 scatole;
- sei persone: p1 necessita di cibo o attrezzi, p2 di medicine, p3 di medicine, p4 di cibo e medicine, p5 e p6 necessitano di tutto;
- p1 e p2 sono nello stesso luogo; gli altri in luoghi diversi.

Obiettivo: Tutte le persone ferite devono ricevere gli oggetti di cui hanno bisogno.

Per la seconda istanza è stato utilizzato un peso pari a 5.

```

1 Starting search... A* search succeeded
2 Number of states evaluated : 3674
3 found plan as follows:
4
5 00: (          fill a1 b1 food depot) [0]
6 01: (      load a1 c1 place1 b1 depot) [0]
7 02: (          move a1 c1 depot l1) [0]
8 03: (      unload a1 c1 place1 b1 l1) [0]
9 04: (emptyatleastone a1 b1 food p1 l1) [0]
10 05: (      fill a2 b3 medicine depot) [0]
11 06: (          move a1 c1 l1 depot) [0]
12 07: (      load a1 c1 place1 b3 depot) [0]

```

```

13 08: (          move a1 c1 depot l1) [0]
14 09: (      unload a1 c1 place1 b3 l1) [0]
15 10: (      empty a1 b3 medicine p2 l1) [0]
16 11: (      load a1 c1 place1 b3 l1) [0]
17 12: (          move a1 c1 l1 depot) [0]
18 13: (          fill a1 b2 food depot) [0]
19 14: (      load a1 c2 place3 b2 depot) [0]
20 15: (          move a1 c2 depot l3) [0]
21 16: (      unload a1 c2 place3 b2 l3) [0]
22 17: (          empty a1 b2 food p4 l3) [0]
23 18: (      load a1 c2 place3 b2 l3) [0]
24 19: (          move a1 c2 l3 depot) [0]
25 20: (      unload a1 c1 place1 b3 depot) [0]
26 21: (          fill a1 b3 food depot) [0]
27 22: (      load a1 c1 place1 b3 depot) [0]
28 23: (          move a1 c1 depot l4) [0]
29 24: (      unload a1 c1 place1 b3 l4) [0]
30 25: (          empty a1 b3 food p5 l4) [0]
31 26: (      load a1 c1 place2 b3 l4) [0]
32 27: (          move a1 c1 l4 depot) [0]
33 28: (      unload a1 c2 place3 b2 depot) [0]
34 29: (          fill a1 b2 food depot) [0]
35 30: (      load a1 c2 place3 b2 depot) [0]
36 31: (          move a1 c2 depot l5) [0]
37 32: (      unload a1 c2 place3 b2 l5) [0]
38 33: (          empty a1 b2 food p6 l5) [0]
39 34: (      load a1 c2 place3 b2 l5) [0]
40 35: (          move a1 c2 l5 depot) [0]
41 36: (      unload a1 c1 place2 b3 depot) [0]
42 37: (          fill a1 b3 medicine depot) [0]
43 38: (      load a1 c1 place1 b3 depot) [0]
44 39: (          move a1 c1 depot l2) [0]
45 40: (      unload a1 c1 place1 b3 l2) [0]
46 41: (      empty a1 b3 medicine p3 l2) [0]
47 42: (      load a1 c1 place1 b3 l2) [0]
48 43: (          move a1 c1 l2 depot) [0]
49 44: (      unload a1 c2 place3 b2 depot) [0]

```

```

50 45: (          fill a1 b2 tools depot) [0]
51 46: (      load a1 c2 place4 b2 depot) [0]
52 47: (          move a1 c2 depot 14) [0]
53 48: (      unload a1 c2 place4 b2 14) [0]
54 49: (      empty a1 b2 tools p5 14) [0]
55 50: (      load a1 c2 place3 b2 14) [0]
56 51: (          move a1 c2 14 depot) [0]
57 52: (  unload a1 c1 place1 b3 depot) [0]
58 53: (          fill a1 b3 tools depot) [0]
59 54: (      load a1 c1 place1 b3 depot) [0]
60 55: (          move a1 c1 depot 15) [0]
61 56: (      unload a1 c1 place1 b3 15) [0]
62 57: (      empty a1 b3 tools p6 15) [0]
63 58: (  unload a2 c2 place3 b2 depot) [0]
64 59: (          fill a2 b2 medicine depot) [0]
65 60: (      load a2 c2 place3 b2 depot) [0]
66 61: (          move a2 c2 depot 13) [0]
67 62: (      unload a2 c2 place3 b2 13) [0]
68 63: (      load a1 c1 place1 b3 15) [0]
69 64: (          move a1 c1 15 depot) [0]
70 65: (      empty a2 b2 medicine p4 13) [0]
71 66: (  unload a1 c1 place1 b3 depot) [0]
72 67: (          fill a1 b3 medicine depot) [0]
73 68: (      load a1 c1 place1 b3 depot) [0]
74 69: (          move a1 c1 depot 14) [0]
75 70: (      unload a1 c1 place1 b3 14) [0]
76 71: (      empty a1 b3 medicine p5 14) [0]
77 72: (      load a1 c1 place1 b3 14) [0]
78 73: (          move a1 c1 14 depot) [0]
79 74: (  unload a1 c1 place1 b3 depot) [0]
80 75: (          fill a1 b3 medicine depot) [0]
81 76: (      load a1 c1 place1 b3 depot) [0]
82 77: (          move a1 c1 depot 15) [0]
83 78: (      unload a1 c1 place1 b3 15) [0]
84 79: (      empty a1 b3 medicine p6 15) [0]
85
86 time spent:          0,01 seconds parsing

```

```

87         0,06 seconds encoding
88         0,78 seconds searching
89         0,84 seconds total time
90
91 memory used:      1,87 MBytes for problem representation
92                  0,00 MBytes for searching
93                  1,87 MBytes total

```

E' possibile vedere come l'applicazione del pruning stocastico nella seconda istanza riduca il numero di stati valutati.

```

1 Starting search... A* search succeeded
2 Number of states evaluated : 1768

```

2.2.3 Istanza 3

Infine, è riportata di seguito la definizione del terzo problem file.

```

1 (define
2   (problem problem3)
3   (:domain emergency)
4
5   (:objects
6     a1 a2 - agent
7     c1 c2 - carrier
8     depot l1 l2 l3 l4 l5 l6 l7 - location
9     food medicine tools - content
10    place1 place2 place3 place4 - carrierplace
11    b1 b2 b3 b4 - box
12    p1 p2 p3 p4 p5 p6 p7 p8 - person
13  )
14
15  (:init
16    (empty b1)
17    (empty b2)
18    (empty b3)
19    (empty b4)
20
21    (availablePlace place1)

```

```

22      (availablePlace place2)
23      (availablePlace place3)
24      (availablePlace place4)
25
26      (at a1 depot)
27      (at a2 depot)
28      (at c1 depot)
29      (at c2 depot)
30      (at b1 depot)
31      (at b2 depot)
32      (at b3 depot)
33      (at b4 depot)
34      (at food depot)
35      (at medicine depot)
36      (at tools depot)
37      (at p1 l1) ;ci serve garantire che p1 e p2 siano
nello stesso luogo e che gli altri siano in luoghi
diversi
38      (at p2 l1)
39      (at p3 l2)
40      (at p4 l3)
41      (at p5 l4)
42      (at p6 l5)
43      (at p7 l6)
44      (at p8 l7)
45
46      (carrierPlace place1 c1)
47      (carrierPlace place2 c1)
48      (carrierPlace place3 c2)
49      (carrierPlace place4 c2)
50
51      (need p1 food) ;p1 needs food or tools
52      (need p1 tools)
53      (need p2 medicine) ; p2 needs medicine
54      (need p3 medicine) ;p3 needs medicine
55      (need p4 medicine) ;p4 need medicine and food
56      (need p4 food)

```

```

57      (need p5 medicine) ;p5, p6, p7 and p8 need everything
58      .
59      (need p5 food)
60      (need p5 tools)
61      (need p6 medicine)
62      (need p6 food)
63      (need p6 tools)
64      (need p7 medicine)
65      (need p7 food)
66      (need p7 tools)
67      (need p8 medicine)
68      (need p8 food)
69      (need p8 tools)
70      (needSomething p1) ;ci basta che abbia uno tra food e
71      tools
72      (needAll p2)
73      (needAll p3)
74      (needAll p4)
75      (needAll p5)
76      (needAll p6)
77      (needAll p7)
78      (needAll p8)
79 )
80
81 (:goal (and
82      (hasSomething p1)
83
84      (has p2 medicine)
85
86      (has p3 medicine)
87
88      (has p4 medicine)
89      (has p4 food)
90
91      (has p5 food)

```

```

92     (has p5 medicine)
93     (has p5 tools)
94
95     (has p6 food)
96     (has p6 medicine)
97     (has p6 tools)
98
99     (has p7 food)
100    (has p7 medicine)
101    (has p7 tools)
102
103    (has p8 food)
104    (has p8 medicine)
105    (has p8 tools))
106 )
107
108 )

```

Condizioni iniziali:

- quattro scatole;
- otto persone: p1 necessita di cibo o attrezzi, p2 di medicine, p3 di medicine, p4 necessita di medicine e cibo, p5, p6, p7 e p8 necessitano di tutto;
- p1 e p2 si trovano nello stesso luogo, gli altri in luoghi diversi.

Obiettivo: Tutte le persone ferite devono ricevere gli oggetti di cui hanno bisogno.

Per la terza istanza è stato utilizzato un peso pari a 20.

```

1 Starting search... A* search succeeded
2 Number of states evaluated : 12030
3 found plan as follows:
4
5 000: (          fill a1 b3 medicine depot) [0]
6 001: (          load a1 c2 place4 b3 depot) [0]
7 002: (          move a1 c2 depot l1) [0]

```

```

8 003: (      unload a1 c2 place4 b3 l1) [0]
9 004: (      empty a1 b3 medicine p2 l1) [0]
10 005: (      fill a2 b4 food depot) [0]
11 006: (      move a1 c2 l1 depot) [0]
12 007: (      load a1 c2 place4 b4 depot) [0]
13 008: (      move a1 c2 depot l1) [0]
14 009: (      unload a1 c2 place4 b4 l1) [0]
15 010: (emptyatleastone a1 b4 food p1 l1) [0]
16 011: (      fill a2 b2 food depot) [0]
17 012: (      move a1 c2 l1 depot) [0]
18 013: (      load a1 c2 place4 b2 depot) [0]
19 014: (      move a1 c2 depot l3) [0]
20 015: (      unload a1 c2 place4 b2 l3) [0]
21 016: (      empty a1 b2 food p4 l3) [0]
22 017: (      load a1 c2 place3 b2 l3) [0]
23 018: (      move a1 c2 l3 depot) [0]
24 019: (      fill a1 b1 food depot) [0]
25 020: (      load a1 c1 place1 b1 depot) [0]
26 021: (      move a1 c1 depot l4) [0]
27 022: (      unload a1 c1 place1 b1 l4) [0]
28 023: (      empty a1 b1 food p5 l4) [0]
29 024: (      load a1 c1 place1 b1 l4) [0]
30 025: (      move a1 c1 l4 depot) [0]
31 026: (      unload a1 c1 place1 b1 depot) [0]
32 027: (      fill a1 b1 food depot) [0]
33 028: (      load a1 c1 place1 b1 depot) [0]
34 029: (      move a1 c1 depot l5) [0]
35 030: (      unload a1 c1 place1 b1 l5) [0]
36 031: (      empty a1 b1 food p6 l5) [0]
37 032: (      load a1 c1 place1 b1 l5) [0]
38 033: (      move a1 c1 l5 depot) [0]
39 034: (      unload a1 c2 place3 b2 depot) [0]
40 035: (      fill a1 b2 food depot) [0]
41 036: (      load a1 c2 place3 b2 depot) [0]
42 037: (      move a1 c2 depot l6) [0]
43 038: (      unload a1 c2 place3 b2 l6) [0]
44 039: (      empty a1 b2 food p7 l6) [0]

```

```

45 040: (          load a1 c2 place3 b2 16) [0]
46 041: (          move a1 c2 16 depot) [0]
47 042: (    unload a1 c2 place3 b2 depot) [0]
48 043: (          fill a1 b2 tools depot) [0]
49 044: (          load a1 c2 place4 b2 depot) [0]
50 045: (          move a1 c2 depot 14) [0]
51 046: (          unload a1 c2 place4 b2 14) [0]
52 047: (          empty a1 b2 tools p5 14) [0]
53 048: (          load a1 c2 place3 b2 14) [0]
54 049: (          move a1 c2 14 depot) [0]
55 050: (    unload a1 c1 place1 b1 depot) [0]
56 051: (          fill a1 b1 food depot) [0]
57 052: (          load a1 c1 place1 b1 depot) [0]
58 053: (          move a1 c1 depot 17) [0]
59 054: (          unload a1 c1 place1 b1 17) [0]
60 055: (          empty a1 b1 food p8 17) [0]
61 056: (          load a1 c1 place1 b1 17) [0]
62 057: (          move a1 c1 17 depot) [0]
63 058: (    unload a1 c2 place3 b2 depot) [0]
64 059: (          fill a1 b2 tools depot) [0]
65 060: (          load a1 c2 place4 b2 depot) [0]
66 061: (          move a1 c2 depot 15) [0]
67 062: (          unload a1 c2 place4 b2 15) [0]
68 063: (          empty a1 b2 tools p6 15) [0]
69 064: (          load a1 c2 place3 b2 15) [0]
70 065: (          move a1 c2 15 depot) [0]
71 066: (    unload a1 c1 place1 b1 depot) [0]
72 067: (          fill a1 b1 medicine depot) [0]
73 068: (          load a1 c1 place1 b1 depot) [0]
74 069: (          move a1 c1 depot 12) [0]
75 070: (          unload a1 c1 place1 b1 12) [0]
76 071: (          empty a1 b1 medicine p3 12) [0]
77 072: (          load a1 c1 place1 b1 12) [0]
78 073: (          move a1 c1 12 depot) [0]
79 074: (    unload a1 c2 place3 b2 depot) [0]
80 075: (          fill a1 b2 tools depot) [0]
81 076: (          load a1 c2 place4 b2 depot) [0]

```

```

82 077: (          move a1 c2 depot 16) [0]
83 078: (      unload a1 c2 place4 b2 16) [0]
84 079: (          empty a1 b2 tools p7 16) [0]
85 080: (          load a1 c2 place3 b2 16) [0]
86 081: (          move a1 c2 16 depot) [0]
87 082: (      unload a1 c2 place3 b2 depot) [0]
88 083: (          fill a1 b2 medicine depot) [0]
89 084: (          load a1 c2 place4 b2 depot) [0]
90 085: (          move a1 c2 depot 13) [0]
91 086: (          unload a1 c2 place4 b2 13) [0]
92 087: (          empty a1 b2 medicine p4 13) [0]
93 088: (          load a1 c2 place3 b2 13) [0]
94 089: (          move a1 c2 13 depot) [0]
95 090: (      unload a1 c1 place1 b1 depot) [0]
96 091: (          fill a1 b1 medicine depot) [0]
97 092: (          load a1 c1 place1 b1 depot) [0]
98 093: (          move a1 c1 depot 14) [0]
99 094: (          unload a1 c1 place1 b1 14) [0]
100 095: (          empty a1 b1 medicine p5 14) [0]
101 096: (          load a1 c1 place1 b1 14) [0]
102 097: (          move a1 c1 14 depot) [0]
103 098: (      unload a1 c2 place3 b2 depot) [0]
104 099: (          fill a1 b2 tools depot) [0]
105 100: (          load a1 c2 place4 b2 depot) [0]
106 101: (          move a1 c2 depot 17) [0]
107 102: (          unload a1 c2 place4 b2 17) [0]
108 103: (          empty a1 b2 tools p8 17) [0]
109 104: (      unload a2 c1 place1 b1 depot) [0]
110 105: (          fill a2 b1 medicine depot) [0]
111 106: (          load a2 c1 place1 b1 depot) [0]
112 107: (          move a2 c1 depot 15) [0]
113 108: (          unload a2 c1 place1 b1 15) [0]
114 109: (          load a1 c2 place3 b2 17) [0]
115 110: (          move a1 c2 17 depot) [0]
116 111: (          empty a2 b1 medicine p6 15) [0]
117 112: (      unload a1 c2 place3 b2 depot) [0]
118 113: (          fill a1 b2 medicine depot) [0]

```

```

119 114: (      load a1 c2 place3 b2 depot) [0]
120 115: (      move a1 c2 depot 16) [0]
121 116: (      unload a1 c2 place3 b2 16) [0]
122 117: (      empty a1 b2 medicine p7 16) [0]
123 118: (      load a1 c2 place3 b2 16) [0]
124 119: (      move a1 c2 16 depot) [0]
125 120: (      unload a1 c2 place3 b2 depot) [0]
126 121: (      fill a1 b2 medicine depot) [0]
127 122: (      load a1 c2 place3 b2 depot) [0]
128 123: (      move a1 c2 depot 17) [0]
129 124: (      unload a1 c2 place3 b2 17) [0]
130 125: (      empty a1 b2 medicine p8 17) [0]
131
132 time spent:      0,02 seconds parsing
133                0,08 seconds encoding
134                5,06 seconds searching
135                5,15 seconds total time
136
137 memory used:      3,25 MBytes for problem representation
138                0,00 MBytes for searching
139                3,25 MBytes total

```

Nella terza istanza l'applicazione del pruning stocastico genera una drastica riduzione del numero di stati valutati e del tempo impiegato, nonostante l'aumento del numero di azioni del piano (142 invece che 125).

```

1 Starting search... A* search succeeded
2 Number of states evaluated : 4597
3 found plan as follows:
4
5 000: (      fill a1 b1 food depot) [0]
6 001: (      load a2 c2 place3 b1 depot) [0]
7 002: (      move a1 c2 depot 16) [0]
8 003: (      unload a1 c2 place3 b1 16) [0]
9 004: (      empty a1 b1 food p7 16) [0]
10 005: (      fill a2 b2 medicine depot) [0]
11 006: (      move a1 c2 16 depot) [0]

```

```

12 007: (      load a1 c1 place2 b2 depot) [0]
13 008: (      move a1 c1 depot 13) [0]
14 009: (      unload a1 c1 place2 b2 13) [0]
15 010: (      empty a1 b2 medicine p4 13) [0]
16 011: (      fill a2 b3 food depot) [0]
17 012: (      load a2 c2 place3 b3 depot) [0]
18 013: (      move a1 c1 13 depot) [0]
19 014: (      move a1 c2 depot 13) [0]
20 015: (      unload a1 c2 place3 b3 13) [0]
21 016: (      load a1 c2 place4 b2 13) [0]
22 017: (      empty a1 b3 food p4 13) [0]
23 018: (      move a1 c2 13 depot) [0]
24 019: (      fill a2 b4 medicine depot) [0]
25 020: (      load a1 c1 place1 b4 depot) [0]
26 021: (      move a1 c1 depot 15) [0]
27 022: (      unload a1 c1 place1 b4 15) [0]
28 023: (      empty a1 b4 medicine p6 15) [0]
29 024: (      load a1 c1 place2 b4 15) [0]
30 025: (      move a1 c1 15 11) [0]
31 026: (      move a1 c1 11 depot) [0]
32 027: (      unload a1 c2 place4 b2 depot) [0]
33 028: (      fill a1 b2 tools depot) [0]
34 029: (      load a1 c2 place3 b2 depot) [0]
35 030: (      move a1 c2 depot 16) [0]
36 031: (      load a1 c2 place4 b1 16) [0]
37 032: (      unload a1 c2 place3 b2 16) [0]
38 033: (      unload a1 c2 place4 b1 16) [0]
39 034: (      unload a2 c1 place2 b4 depot) [0]
40 035: (      load a1 c2 place4 b1 16) [0]
41 036: (      empty a1 b2 tools p7 16) [0]
42 037: (      move a1 c2 16 depot) [0]
43 038: (      fill a1 b4 medicine depot) [0]
44 039: (      load a1 c1 place1 b4 depot) [0]
45 040: (      move a2 c1 depot 16) [0]
46 041: (      unload a2 c1 place1 b4 16) [0]
47 042: (      empty a2 b4 medicine p7 16) [0]
48 043: (      load a2 c1 place1 b4 16) [0]

```

```

49 044: (          move a2 c1 l6 depot) [0]
50 045: (    unload a2 c1 place1 b4 depot) [0]
51 046: (          fill a1 b4 food depot) [0]
52 047: (    load a1 c1 place1 b4 depot) [0]
53 048: (          move a1 c1 depot l1) [0]
54 049: (    unload a1 c1 place1 b4 l1) [0]
55 050: (emptyatleastone a1 b4 food p1 l1) [0]
56 051: (    load a1 c1 place1 b4 l1) [0]
57 052: (          move a1 c1 l1 depot) [0]
58 053: (    unload a1 c1 place1 b4 depot) [0]
59 054: (    fill a1 b4 medicine depot) [0]
60 055: (    load a1 c1 place1 b4 depot) [0]
61 056: (          move a1 c1 depot l7) [0]
62 057: (          move a1 c1 l7 l2) [0]
63 058: (    unload a1 c1 place1 b4 l2) [0]
64 059: (    empty a1 b4 medicine p3 l2) [0]
65 060: (    load a1 c1 place2 b4 l2) [0]
66 061: (          move a1 c1 l2 depot) [0]
67 062: (    unload a1 c1 place2 b4 depot) [0]
68 063: (    fill a1 b4 food depot) [0]
69 064: (    load a1 c1 place1 b4 depot) [0]
70 065: (          move a1 c1 depot l4) [0]
71 066: (    unload a2 c2 place4 b1 depot) [0]
72 067: (    unload a1 c1 place1 b4 l4) [0]
73 068: (    empty a1 b4 food p5 l4) [0]
74 069: (    load a2 c2 place3 b1 depot) [0]
75 070: (          move a1 c1 l4 l3) [0]
76 071: (    load a1 c1 place1 b3 l3) [0]
77 072: (          move a1 c1 l3 depot) [0]
78 073: (    unload a1 c1 place1 b3 depot) [0]
79 074: (    fill a1 b3 medicine depot) [0]
80 075: (    load a1 c1 place1 b3 depot) [0]
81 076: (          move a2 c1 depot l4) [0]
82 077: (    unload a2 c1 place1 b3 l4) [0]
83 078: (    unload a1 c2 place3 b1 depot) [0]
84 079: (    load a2 c1 place1 b4 l4) [0]
85 080: (    empty a2 b3 medicine p5 l4) [0]

```

```

86 081: (          move a2 c1 l4 depot) [0]
87 082: (          fill a1 b1 food depot) [0]
88 083: (    unload a1 c1 place1 b4 depot) [0]
89 084: (          load a2 c2 place3 b1 depot) [0]
90 085: (          move a2 c2 depot 17) [0]
91 086: (          unload a2 c2 place3 b1 17) [0]
92 087: (          empty a2 b1 food p8 17) [0]
93 088: (          load a2 c2 place3 b1 17) [0]
94 089: (          move a2 c2 17 depot) [0]
95 090: (          fill a1 b4 tools depot) [0]
96 091: (          load a1 c1 place2 b4 depot) [0]
97 092: (          move a2 c1 depot 14) [0]
98 093: (          unload a2 c1 place2 b4 14) [0]
99 094: (          empty a2 b4 tools p5 14) [0]
100 095: (          load a2 c1 place2 b4 14) [0]
101 096: (          move a2 c1 l4 depot) [0]
102 097: (    unload a1 c1 place2 b4 depot) [0]
103 098: (          fill a1 b4 food depot) [0]
104 099: (          load a1 c1 place1 b4 depot) [0]
105 100: (          move a1 c1 depot 15) [0]
106 101: (          unload a1 c1 place1 b4 15) [0]
107 102: (          move a1 c1 15 depot) [0]
108 103: (          move a2 c1 depot 11) [0]
109 104: (          move a2 c1 l1 15) [0]
110 105: (          empty a2 b4 food p6 15) [0]
111 106: (          load a2 c1 place1 b4 15) [0]
112 107: (          move a2 c1 15 depot) [0]
113 108: (    unload a1 c2 place3 b1 depot) [0]
114 109: (    unload a1 c1 place1 b4 depot) [0]
115 110: (          fill a2 b4 medicine depot) [0]
116 111: (          load a2 c2 place4 b4 depot) [0]
117 112: (          move a2 c2 depot 17) [0]
118 113: (          move a2 c2 17 11) [0]
119 114: (          unload a2 c2 place4 b4 11) [0]
120 115: (          empty a2 b4 medicine p2 11) [0]
121 116: (          move a2 c2 l1 14) [0]
122 117: (          load a2 c2 place3 b3 14) [0]

```

```

123 118: (          move a2 c2 l4 depot) [0]
124 119: (          fill a1 b1 tools depot) [0]
125 120: (      load a1 c1 place2 b1 depot) [0]
126 121: (          move a1 c1 depot 15) [0]
127 122: (      unload a1 c1 place2 b1 15) [0]
128 123: (      unload a2 c2 place3 b3 depot) [0]
129 124: (      load a2 c2 place4 b3 depot) [0]
130 125: (          empty a1 b1 tools p6 15) [0]
131 126: (          move a1 c1 l5 l6) [0]
132 127: (      load a1 c1 place1 b2 l6) [0]
133 128: (          move a1 c1 l6 depot) [0]
134 129: (      unload a2 c2 place4 b3 depot) [0]
135 130: (          fill a1 b3 medicine depot) [0]
136 131: (      load a1 c2 place3 b3 depot) [0]
137 132: (          move a1 c2 depot 17) [0]
138 133: (      unload a2 c1 place1 b2 depot) [0]
139 134: (          fill a2 b2 tools depot) [0]
140 135: (      unload a1 c2 place3 b3 l7) [0]
141 136: (          empty a1 b3 medicine p8 l7) [0]
142 137: (          move a1 c2 l7 depot) [0]
143 138: (      load a1 c2 place4 b2 depot) [0]
144 139: (          move a2 c2 depot 17) [0]
145 140: (          move a1 c1 depot 12) [0]
146 141: (          unload a2 c2 place4 b2 l7) [0]
147 142: (          empty a2 b2 tools p8 l7) [0]
148
149 time spent:      0,02 seconds parsing
150                  0,08 seconds encoding
151                  1,45 seconds searching
152                  1,54 seconds total time
153
154 memory used:      3,25 MBytes for problem representation
155                  0,00 MBytes for searching
156                  3,25 MBytes total

```

3 Temporal Planning & Robotics

3.1 Temporal Planning

Il punto 3.1 del progetto richiede di modificare il file di dominio del punto 1 e il file di problema del punto 2.1 in modo da utilizzare le azioni durative. In particolare, richiede che le varie azioni abbiano una durata proporzionale ai pesi degli oggetti coinvolti.

3.1.1 Modifiche al file di dominio

Per svolgere questo compito, il primo passo è stato modificare il file di dominio creato per il task 1. In particolare, sono stati introdotti i fluents per modellare i pesi degli oggetti, dai quali dipendono le durate delle azioni. Inoltre, le azioni presenti nel dominio sono state convertite in durative-actions.

Dopo aver aggiunto i requirements necessari, abbiamo innanzitutto aggiunto delle functions per modellare i pesi dei vari oggetti presenti e per definire la durata delle varie azioni. Operando in questo modo è possibile scegliere i pesi degli oggetti e le durate delle azioni a proprio piacimento impostandole opportunamente nel file di problema. Abbiamo quindi aggiunto le seguenti functions:

- **content_weight**: definisce il peso del contenuto delle scatole.
- **box_weight**: definisce il peso della scatola. Si assume che il peso della scatola è dato dal peso della scatola vuota più i pesi degli oggetti che vengono inseriti al suo interno;
- **carrier_weight**: definisce il peso del carrello. Si assume che il peso del carrello è dato dal peso del carrello scarico più i pesi delle scatole che vengono caricati sul carrello stesso;
- **fill_duration**: indica la durata base dell'azione di riempimento della scatola. Si assume che la durata dell'azione fill sia propor-

zionale al peso dell'oggetto con cui riempire la scatola, cioè è data da: $(* (\text{content_weight } ?c) (\text{fill_duration}))$;

- **move_duration**: indica la durata base dell'azione move. Si assume che la durata dell'azione move sia proporzionale al peso del carrello spostato, cioè è data da: $(* (\text{carrie_weight } ?c) (\text{move_duration}))$;
- **load_duration**: indica la durata base delle azioni load e unload. Si assume che la durata di queste azioni sia proporzionale al peso della scatola caricata/scaricata, cioè è data da: $(* (\text{box_weight } ?c) (\text{load_duration}))$;
- **empty_duration**: indica la durata base delle azioni empty e emptyAtLeastOne. Si assume che la durata di queste azioni sia proporzionale al peso dell'oggetto tolto dalla scatola, cioè è data da: $(* (\text{content_weight } ?c) (\text{empty_duration}))$;

Abbiamo anche introdotto un nuovo predicato chiamato freeAgent per indicare se l'agente è attualmente impegnato nell'esecuzione di un'azione. Questo predicato è necessario per garantire che un agente non esegua più azioni simultaneamente.

Le modifiche alla prima parte del file sono riportate di seguito. I types sono rimasti gli stessi, così come i restanti predicati.

```
1 (define
2   (domain temporal_emergency)
3   (:requirements :strips :typing :durative-actions :
4     fluents)
5   (:types
6     location - object
7     carrierplace - object
8     locatable - object
9     box - locatable
```

```

10         agent - locatable
11         content - locatable
12         carrier - locatable
13         person - locatable
14     )
15
16     (:predicates
17         (at ?o - locatable ?l - location)
18         (empty ?b - box)
19         (inBox ?b - box ?c - content)
20         (boxOnPlace ?b - box ?p - carrierplace)
21         (carrierPlace ?p - carrierplace ?c - carrier)
22         (availablePlace ?p - carrierplace)
23         (has ?p - person ?c - content)
24         (hasSomething ?p - person)
25         (need ?p - person ?c - content)
26         (needAll ?p - person)
27         (needSomething ?p - person)
28         (freeAgent ?g - agent)
29     )
30
31     (:functions
32         (content_weight ?c - content)
33         (box_weight ?b - box)
34         (carrier_weight ?c - carrier)
35         (fill_duration)
36         (move_duration)
37         (load_duration)
38         (empty_duration)
39     )

```

Infine, le azioni sono state convertite in azioni durative. In particolare, abbiamo specificato condizioni ed effetti utilizzando i costrutti `at start`, `over all` e `at end` e prestando attenzione a non utilizzare l'operatore `not` nelle precondizioni delle azioni per evitare problemi con il planner. Le condizioni e gli effetti delle azioni rimangono sostanzialmente gli stessi, con alcune eccezioni: nelle condizioni è necessario che

l'agente sia libero all'inizio dell'azione, mentre negli effetti si specifica che l'agente diventa occupato all'inizio dell'azione e torna libero al termine dell'azione. Le durate delle azioni sono state definite come riportato sopra. Inoltre, abbiamo gestito opportunamente l'incremento e il decremento delle funzioni che mantengono i pesi dei vari oggetti utilizzando i costrutti `increase` e `decrease`. Le modifiche alle azioni sono le seguenti:

```

1      (:durative-action fill
2
3          :parameters (?a - agent ?b - box ?c - content ?l -
4              location)
5
6          :duration (= ?duration (* (content_weight ?c) (
7              fill_duration)))
8
9          :condition (and
10
11              (at start (freeAgent ?a))
12              (at start (empty ?b))
13              (over all (at ?a ?l))
14              (over all (at ?b ?l))
15              (over all (at ?c ?l))
16
17              )
18
19          :effect (and
20
21              (at start (not(freeAgent ?a)))
22              (at start (not(empty ?b)))
23              (at end (inBox ?b ?c))
24              (at end (freeAgent ?a))
25              (at end (increase (box_weight ?b)
26                  (content_weight ?c)))
27
28              )
29
30      )
31
32      (:durative-action move
33
34          :parameters (?a - agent ?c - carrier ?from ?to -
35              location)
36
37          :duration (= ?duration (* (carrier_weight ?c) (
38              move_duration)))
39
40          :condition (and
41
42              (at start (freeAgent ?a))

```

```

27         (at start (at ?a ?from))
28         (at start (at ?c ?from))
29     )
30     :effect (and
31         (at start (not(freeAgent ?a)))
32         (at start (not(at ?a ?from)))
33         (at start (not(at ?c ?from)))
34         (at end (at ?a ?to))
35         (at end (at ?c ?to))
36         (at end (freeAgent ?a))
37     )
38 )
39
40 (:durative-action load
41     :parameters (?a - agent ?c - carrier ?p -
42     carrierplace
43         ?b - box ?l - location)
44     :duration (= ?duration (* (box_weight ?b) (
45     load_duration)))
46     :condition (and
47         (at start (freeAgent ?a))
48         (over all (at ?a ?l))
49         (over all (at ?c ?l))
50         (over all (at ?b ?l))
51         (over all (carrierPlace ?p ?c))
52         (at start (availablePlace ?p))
53     )
54     :effect (and
55         (at start (not(freeAgent ?a)))
56         (at start (not(availablePlace ?p
57         )))
58         (at end (not(at ?b ?l)))
59         (at end (boxOnPlace ?b ?p))
60         (at end (freeAgent ?a))
61         (at end (increase (
62         carrier_weight ?c)
63         (box_weight ?b)))

```

```

60         )
61
62     )
63
64     (:durative-action unload
65         :parameters (?a - agent ?c - carrier ?p
66         carrierplace
67             ?b - box ?l - location)
68         :duration (= ?duration (* (box_weight ?b) (
69         load_duration)))
70         :condition (and
71             (at start (freeAgent ?a))
72             (at start (boxOnPlace ?b ?p))
73             (at start (not(availablePlace ?p
74             )))
75             (over all (at ?a ?l))
76             (over all (at ?c ?l))
77             (over all (carrierPlace ?p ?c))
78         )
79     )
80     :effect (and
81         (at start (not(freeAgent ?a)))
82         (at start(at ?b ?l))
83         (at start (availablePlace ?p))
84         (at end (not(boxOnPlace ?b ?p)))
85         (at end (freeAgent ?a))
86         (at end(decrease (carrier_weight
87         ?c)
88             (box_weight ?b)
89         ))
90     )
91 )
92
93 (:durative-action empty
94     :parameters (?a - agent ?b - box ?c - content ?p -

```

```

person
92         ?l - location)
93     :duration (= ?duration (* (content_weight ?c) (
empty_duration)))
94     :condition (and
95         (at start (freeAgent ?a))
96         (at start (inBox ?b ?c))
97         (at start (need ?p ?c))
98         (over all (at ?a ?l))
99         (over all (at ?b ?l))
100        (over all (at ?p ?l))
101        (over all (needAll ?p))
102    )
103    :effect    (and
104        (at start (not(freeAgent ?a)))
105        (at start (empty ?b))
106        (at start (not (need ?p ?c)))
107        (at end (not(inBox ?b ?c)))
108        (at end (has ?p ?c))
109        (at end (freeAgent ?a))
110        (at end (decrease (box_weight ?b)
111            (content_weight ?c)))
112    )
113 )
114
115
116 (:durative-action emptyAtLeastOne
117     :parameters (?a - agent ?b - box ?c -
content ?p    person
118         ?l - location)
119     :duration (= ?duration (* (content_weight ?c
)
120         (empty_duration)))
121     :condition (and
122         (at start (freeAgent ?a))
123         (at start (inBox ?b ?c))
124         (at start (need ?p ?c))

```

```

125         (over all (at ?a ?l))
126         (over all (at ?b ?l))
127         (over all (at ?p ?l))
128         (over all (needSomething ?p)
129     )
130     :effect (and
131         (at start (not(freeAgent ?a)
132         (at start (empty ?b))
133         (at start (not (need ?p ?c))
134     )
135         (at start (not (
136         needSomething ?p)))
137         (at end (not(inBox ?b ?c)))
138         (at end (hasSomething ?p))
139         (at end (freeAgent ?a))
140         (at end (decrease (
141         box_weight ?b)
142         (
143         content_weight ?c)))
144     )
145 )

```

3.1.2 Modifiche al file di problema

Successivamente, abbiamo aggiornato il file del problema dell'istanza 2.1 per allinearlo al nuovo file di dominio. Il file rimane molto simile a quello definito per l'istanza 2.1, sono state apportate solo alcune modifiche: abbiamo inizializzato le funzioni relative ai pesi degli oggetti e alle durate delle azioni, e impostato il predicato `freeAgent` per l'agente `a`.

```

1 . . .
2
3 (:init
4     (= (content_weight drug) 1)

```

```

5      (= (content_weight food) 2)
6      (= (content_weight tools) 3)
7
8      (= (box_weight b1) 1)
9      (= (box_weight b2) 1)
10     (= (box_weight b3) 1)
11     (= (box_weight b4) 1)
12     (= (box_weight b5) 1)
13
14     (= (carrier_weight c) 3)
15
16     (= (fill_duration) 1)
17     (= (move_duration) 5)
18     (= (load_duration) 2)
19     (= (empty_duration) 3)
20
21     . . .
22
23     (freeAgent a)
24
25     . . .
26 )
27 . . .

```

In particolare, abbiamo fissato un peso pari rispettivamente a 1, 2, 3 agli oggetti drug, food e tools come richiesto dalla traccia. Abbiamo supposto che la scatola vuota abbia un peso pari a 1, mentre il carrello vuoto abbia un peso pari a 3. Le durate base delle azioni le abbiamo poste pari a 1 per l'azione fill, a 5 per l'azione move, a 2 per le azioni load e unload e a 3 per le azioni empty e emptyAtLeastOne.

3.1.3 Ottenimento del piano

Per risolvere il problema abbiamo utilizzato uno dei planner messi a disposizione dalla libreria planutils, in particolare abbiamo scelto di utilizzare il planner Lpg-td . Per generare il piano abbiamo quindi chia-

mato il risolutore lpg-td passandogli il file di dominio e il file di problema definiti in precedenza, ottenendo il seguente piano:

```
1 ; Version LPG-td-1.4
2 ; Seed 43193157
3 ; Command line: ./lpg-td -o domain_extended.pddl -f
4   problem1_extended.pddl -out plan_problem1 -quality
5 ; Problem problem1_extended.pddl
6 ; Time 12.26
7 ; Plan generation time 5.12
8 ; Search time 5.12
9 ; Parsine time 0.00
10 ; Mutex time 0.00
11 ; MetricValue 18.00
12
13 0.0003:  (FILL A B2 DRUG DEPOT) [1.0000]
14 1.0005:  (LOAD A C PLACE1 B2 DEPOT) [4.0000]
15 5.0008:  (FILL A B5 DRUG DEPOT) [1.0000]
16 6.0010:  (LOAD A C PLACE2 B5 DEPOT) [4.0000]
17 10.0013: (FILL A B4 FOOD DEPOT) [2.0000]
18 12.0015: (FILL A B3 FOOD DEPOT) [2.0000]
19 14.0017: (LOAD A C PLACE3 B4 DEPOT) [6.0000]
20 20.0020: (LOAD A C PLACE4 B3 DEPOT) [6.0000]
21 26.0023: (MOVE A C DEPOT L1) [65.0000]
22 91.0025: (UNLOAD A C PLACE1 B2 L1) [4.0000]
23 95.0027: (EMPTY A B2 DRUG P1 L1) [3.0000]
24 98.0030: (UNLOAD A C PLACE2 B5 L1) [4.0000]
25 102.0033: (EMPTY A B5 DRUG P2 L1) [3.0000]
26 105.0035: (UNLOAD A C PLACE4 B3 L1) [6.0000]
27 111.0038: (EMPTY A B3 FOOD P1 L1) [6.0000]
28 117.0040: (MOVE A C L1 L2) [30.0000]
29 147.0043: (UNLOAD A C PLACE3 B4 L2) [6.0000]
30 153.0045: (EMPTY A B4 FOOD P3 L2) [6.0000]
```

Il piano generato sembra essere molto buono. Le azioni eseguite sono coerenti sia con le condizioni iniziali sia con gli obiettivi. Il numero di azioni eseguite sembra essere il minimo possibile, in effetti non vengono eseguite azioni inutili. L'agente riempie quattro scatole con i beni richie-

sti dalle persone, le carica tutte sul carrello, si sposta alla location l1 e consegna i beni alle persone p1 e p2, per poi spostarsi alla location l2 e consegnare i beni alla persona p3. Anche le durate delle azioni sono coerenti con il modo in cui sono state definite. Infine, il tempo complessivo di esecuzione del piano risulta essere abbastanza basso.

3.2 Robotics Planning

L'ultimo task consiste nell'implementare il problema definito nel punto 3.1 all'interno di PlanSys2. A tal fine è stato creato il workspace `robotic_planning_ws` all'interno del workspace di PlanSys2.

Tale workspace deve avere una determinata struttura e deve includere determinati file. All'interno della cartella `robotic_planning_ws` sono presenti:

- **la cartella `pddl`**, contenente il file `pddl` di dominio;
- **la cartella `launch`**, contenente:
 - il file python `launch.py` usato per avviare i nodi ros;
 - il file `plan.txt` contenente il piano generato al precedente punto 3.1;
 - un file `commands.txt`, usato per settare le istanze e i predicati del problema;
- **la cartella `src`**, contenente i file delle fake action;
- **il file `CMakeList.txt`**, necessario per la build della cartella;
- **il file `package.xml`**, usato per settare le dipendenze necessarie.

Descriviamo quindi i file nel dettaglio.

3.2.1 File del workspace

Abbiamo innanzitutto definito il file contenente il dominio del problema. Al fine di evitare errori nei comandi del terminale di PlanSys2, sono state apportate le seguenti modifiche al file di dominio:

- L'intero file è stato modificato per evitare l'uso di lettere maiuscole. Di conseguenza, tutte le lettere maiuscole sono state convertite in minuscole.
- I tipi definiti in precedenza sono stati trasformati in sottotipi del nuovo tipo "localizable" a causa dei problemi riscontrati con l'utilizzo del tipo "object".
- È stato modificato il nome del predicato "empty", il quale indica se una scatola è vuota o meno, poiché sono stati riscontrati problemi con l'omonima azione "empty", che effettua lo scaricamento della scatola e la consegna del contenuto alla persona. Il nome del predicato è stato quindi cambiato in "emptybox".

Il file di dominio modificato è il seguente:

```
1 (define
2   (domain temporal_emergency)
3   (:requirements :strips :typing :durative-actions :fluents)
4
5   (:types
6     location - localizable
7     carrierplace - localizable
8     agent - localizable
9     content - localizable
10    box - localizable
11    carrier - localizable
12    person - localizable
13  )
14
15  (:functions
```

```

16 (content_weight ?c - content)
17 (box_weight ?b - box)
18 (carrier_weight ?c - carrier)
19 (fill_duration)
20 (move_duration)
21 (load_duration)
22 (empty_duration)
23 )
24
25 (:predicates
26 (at ?o - localizable ?l - location)
27 (emptybox ?b - box)
28 (inbox ?b - box ?c - content)
29 (boxonplace ?b - box ?p - carrierplace)
30 (carrierplace ?p - carrierplace ?c - carrier)
31 (availableplace ?p - carrierplace)
32 (has ?p - person ?c - content)
33 (hassomething ?p - person)
34 (need ?p - person ?c - content)
35 (needall ?p - person)
36 (needsomething ?p - person)
37 (freeagent ?g - agent)
38 )
39
40 (:durative-action fill
41 :parameters (?a - agent ?b - box ?c - content ?l -
42   location)
43 :duration (= ?duration (* (content_weight ?c) (
44   fill_duration)))
45 :condition (and
46   (at start (freeagent ?a))
47   (at start (emptybox ?b))
48   (over all (at ?a ?l))
49   (over all (at ?b ?l))
50   (over all (at ?c ?l))
51 )
52 :effect (and

```

```

51   (at start (not (freeagent ?a)))
52   (at start (not (emptybox ?b)))
53   (at end (inbox ?b ?c))
54   (at end (increase (box_weight ?b) (content_weight ?c)))
55   (at end (freeagent ?a))
56 )
57 )
58
59 (:durative-action move
60 :parameters (?a - agent ?c - carrier ?from - location ?to
61              - location)
62 :duration (= ?duration (* (carrier_weight ?c) (
63                           move_duration)))
64 :condition (and
65   (at start (freeagent ?a))
66   (at start (at ?a ?from))
67   (at start (at ?c ?from))
68 )
69 :effect (and
70   (at start (not (freeagent ?a)))
71   (at start (not (at ?a ?from)))
72   (at start (not (at ?c ?from)))
73   (at end (at ?a ?to))
74   (at end (at ?c ?to))
75   (at end (freeagent ?a))
76 )
77 )
78
79 (:durative-action empty
80 :parameters (?a - agent ?b - box ?c - content ?p - person
81              ?l - location)
82 :duration (= ?duration (* (content_weight ?c) (
83                           empty_duration)))
84 :condition (and
85   (at start (freeagent ?a))
86   (at start (inbox ?b ?c))
87   (at start (need ?p ?c))

```

```

84   (over all (at ?a ?l))
85   (over all (at ?b ?l))
86   (over all (at ?p ?l))
87   (over all (needall ?p))
88 )
89 :effect (and
90   (at start (not(freeagent ?a)))
91   (at start (emptybox ?b))
92   (at start (not (need ?p ?c)))
93   (at end (not (inbox ?b ?c)))
94   (at end (has ?p ?c))
95   (at end (decrease (box_weight ?b) (content_weight ?c)))
96   (at end (freeagent ?a))
97 )
98 )
99
100 (:durative-action load
101   :parameters (?a - agent ?c - carrier ?p - carrierplace ?b
102     - box ?l - location)
103   :duration (= ?duration (* (box_weight ?b) (load_duration))
104     )
105   :condition (and
106     (at start (freeagent ?a))
107     (over all (at ?a ?l))
108     (over all (at ?b ?l))
109     (over all (at ?c ?l))
110     (over all (carrierplace ?p ?c))
111     (at start (avaialableplace ?p))
112   )
113   :effect (and
114     (at start (not (freeagent ?a)))
115     (at start (not (avaialableplace ?p)))
116     (at end (not (at ?b ?l)))
117     (at end (boxonplace ?b ?p))
118     (at end (increase (carrier_weight ?c) (box_weight ?b)))
119     (at end (freeagent ?a))
120   )

```

```

119 )
120
121 (:durative-action unload
122 :parameters (?a - agent ?c - carrier ?p - carrierplace ?b
123   - box ?l - location)
124 :duration (= ?duration (* (box_weight ?b) (load_duration))
125   )
126 :condition (and
127   (at start (freeagent ?a))
128   (at start (boxonplace ?b ?p))
129   (over all (at ?a ?l))
130   (over all (at ?c ?l))
131   (over all (carrierplace ?p ?c))
132 )
133 :effect (and
134   (at start (not (freeagent ?a)))
135   (at start (at ?b ?l))
136   (at start (availableplace ?p))
137   (at end (not (boxonplace ?b ?p)))
138   (at end (decrease (carrier_weight ?c) (box_weight ?b)))
139   (at end (freeagent ?a))
140 )
141 )
142
143 (:durative-action emptyatleastone
144 :parameters (?a - agent ?b - box ?c - content ?p - person
145   ?l - location)
146 :duration (= ?duration (* (content_weight ?c) (
147   empty_duration)))
148 :condition (and
149   (at start (freeagent ?a))
150   (at start (inbox ?b ?c))
151   (at start (need ?p ?c))
152   (over all (at ?a ?l))
153   (over all (at ?b ?l))
154   (over all (at ?p ?l))
155   (over all (needsomething ?p))

```

```

152 )
153 :effect (and
154   (at start (not (freeagent ?a)))
155   (at start (emptybox ?b))
156   (at start (not (need ?p ?c)))
157   (at start (not (needsomething ?p)))
158   (at end (not (inbox ?b ?c)))
159   (at end (hassomething ?p))
160   (at end (decrease (box_weight ?b) (content_weight ?c)))
161   (at end (freeagent ?a))
162 )
163 )
164
165 )

```

Successivamente abbiamo definito il file `launch.py`, un file di configurazione che permette di avviare e configurare i nodi di un sistema basato su ROS. Nel file è stato specificato il modello di dominio PDDL a cui i nodi devono fare riferimento ed è stato definito un nodo per ciascuna azione definita nel file di dominio.

Il file `launch` è il seguente:

```

1  import os
2
3  from ament_index_python.packages import
    get_package_share_directory
4
5  from launch import LaunchDescription
6  from launch.actions import DeclareLaunchArgument,
    IncludeLaunchDescription
7  from launch.launch_description_sources import
    PythonLaunchDescriptionSource
8  from launch.substitutions import LaunchConfiguration
9  from launch_ros.actions import Node
10
11 def generate_launch_description():
12     example_dir = get_package_share_directory('

```

```

robotic_planning_ws') # launch directory
13 namespace = LaunchConfiguration('namespace')
14
15 declare_namespace_cmd = DeclareLaunchArgument(
16     'namespace',
17     default_value='',
18     description='Namespace')
19
20 plansys2_cmd = IncludeLaunchDescription(
21     PythonLaunchDescriptionSource(os.path.join(
22         get_package_share_directory('plansys2_bringup'),
23         'launch',
24         'plansys2_bringup_launch_monolithic.py')),
25     launch_arguments={
26         'model_file': example_dir + '/pddl/domain_extended
.pddl',
27         'namespace': namespace
28     }.items())
29
30 move_cmd = Node(
31     package='robotic_planning_ws',
32     executable='move',
33     name='move',
34     namespace=namespace,
35     output='screen',
36     parameters=[])
37
38 fill_cmd = Node(
39     package='robotic_planning_ws',
40     executable='fill',
41     name='fill',
42     namespace=namespace,
43     output='screen',
44     parameters=[])
45
46 load_cmd = Node(
47     package='robotic_planning_ws',

```

```

48     executable='load',
49     name='load',
50     namespace=namespace,
51     output='screen',
52     parameters=[])
53
54 unload_cmd = Node(
55     package='robotic_planning_ws',
56     executable='unload',
57     name='unload',
58     namespace=namespace,
59     output='screen',
60     parameters=[])
61
62 empty_cmd = Node(
63     package='robotic_planning_ws',
64     executable='empty',
65     name='empty',
66     namespace=namespace,
67     output='screen',
68     parameters=[])
69
70 emptyatleastone_cmd = Node(
71     package='robotic_planning_ws',
72     executable='emptyatleastone',
73     name='emptyatleastone',
74     namespace=namespace,
75     output='screen',
76     parameters=[])
77
78 ld = LaunchDescription()
79
80 ld.add_action(declare_namespace_cmd)
81
82 # Declare the launch options
83 ld.add_action(plansys2_cmd)
84

```

```

85     ld.add_action(move_cmd)
86     ld.add_action(fill_cmd)
87     ld.add_action(load_cmd)
88     ld.add_action(unload_cmd)
89     ld.add_action(empty_cmd)
90     ld.add_action(emptyatleastone_cmd)
91
92     return ld

```

Successivamente abbiamo redatto il file contenente il piano generato nel passo precedente. È stato necessario adattare la sintassi del piano affinché fosse compatibile con POPF, il planner pre-implementato all'interno di PlanSys2.

Il file plan.txt è il seguente:

```

1 0.0003:  (fill a b2 drug depot) [1.0000]
2 1.0005:  (load a c place1 b2 depot) [4.0000]
3 5.0008:  (fill a b5 drug depot) [1.0000]
4 6.0010:  (load a c place2 b5 depot) [4.0000]
5 10.0013: (fill a b4 food depot) [2.0000]
6 12.0015: (fill a b3 food depot) [2.0000]
7 14.0017: (load a c place3 b4 depot) [6.0000]
8 20.0020: (load a c place4 b3 depot) [6.0000]
9 26.0023: (move a c depot l1) [65.0000]
10 91.0025: (unload a c place1 b2 l1) [4.0000]
11 95.0027: (empty a b2 drug p1 l1) [3.0000]
12 98.0030: (unload a c place2 b5 l1) [4.0000]
13 102.0033: (empty a b5 drug p2 l1) [3.0000]
14 105.0035: (unload a c place4 b3 l1) [6.0000]
15 111.0038: (empty a b3 food p1 l1) [6.0000]
16 117.0040: (move a c l1 l2) [30.0000]
17 147.0043: (unload a c place3 b4 l2) [6.0000]
18 153.0045: (empty a b4 food p3 l2) [6.0000]

```

In seguito, abbiamo creato il file commands.txt da utilizzare nel terminale di PlanSys2 per configurare le istanze specifiche del problema:

```

1     set instance a agent
2 set instance c carrier

```

```

3 set instance place1 carrierplace
4 set instance place2 carrierplace
5 set instance place3 carrierplace
6 set instance place4 carrierplace
7 set instance depot location
8 set instance l1 location
9 set instance l2 location
10 set instance p1 person
11 set instance p2 person
12 set instance p3 person
13 set instance b1 box
14 set instance b2 box
15 set instance b3 box
16 set instance b4 box
17 set instance b5 box
18 set instance food content
19 set instance drug content
20 set instance tools content
21
22 set function (= (content_weight drug) 1)
23 set function (= (content_weight food) 2)
24 set function (= (content_weight tools) 3)
25
26 set function (= (box_weight b1) 1)
27 set function (= (box_weight b2) 1)
28 set function (= (box_weight b3) 1)
29 set function (= (box_weight b4) 1)
30 set function (= (box_weight b5) 1)
31
32 set function (= (carrier_weight c) 3)
33
34 set function (= (fill_duration) 1)
35 set function (= (move_duration) 5)
36 set function (= (load_duration) 2)
37 set function (= (empty_duration) 3)
38
39 set predicate (freeagent a)

```

```
40
41 set predicate (emptybox b1)
42 set predicate (emptybox b2)
43 set predicate (emptybox b3)
44 set predicate (emptybox b4)
45 set predicate (emptybox b5)
46
47 set predicate (avaiableplace place1)
48 set predicate (avaiableplace place2)
49 set predicate (avaiableplace place3)
50 set predicate (avaiableplace place4)
51
52 set predicate (at a depot)
53 set predicate (at c depot)
54 set predicate (at b1 depot)
55 set predicate (at b2 depot)
56 set predicate (at b3 depot)
57 set predicate (at b4 depot)
58 set predicate (at b5 depot)
59 set predicate (at food depot)
60 set predicate (at drug depot)
61 set predicate (at tools depot)
62 set predicate (at p1 l1)
63 set predicate (at p2 l1)
64 set predicate (at p3 l2)
65
66 set predicate (carrierplace place1 c)
67 set predicate (carrierplace place2 c)
68 set predicate (carrierplace place3 c)
69 set predicate (carrierplace place4 c)
70
71 set predicate (need p1 food)
72 set predicate (need p1 drug)
73 set predicate (need p2 drug)
74 set predicate (need p3 food)
75
76 set predicate (needall p1)
```

```

77 set predicate (needall p2)
78 set predicate (needall p3)
79
80 set goal ( and (has p1 food) (has p1 drug) (has p2 drug) (
      has p3 food) )

```

In seguito, abbiamo proceduto alla definizione delle fake actions, necessarie per simulare le azioni eseguite dal robot all'interno del sistema gestito da ROS. Queste azioni sono implementate attraverso file C++ situati nella directory src, in particolare è necessario definire un file per ciascuna azione. Visto che i files C++ delle fake actions hanno la stessa struttura, riportiamo solo il contenuto del file relativo alla fake action "move".

```

1   #include <memory>
2   #include <algorithm>
3
4   #include "plansys2_executor/ActionExecutorClient.hpp"
5
6   #include "rclcpp/rclcpp.hpp"
7   #include "rclcpp_action/rclcpp_action.hpp"
8
9   using namespace std::chrono_literals;
10
11
12  class MoveAction : public plansys2::ActionExecutorClient
13  {
14  public:
15      MoveAction()
16      : plansys2::ActionExecutorClient("move", 200ms) // 1'
          azione viene chiamata ogni 200 millisecondi
17      {
18          progress_ = 0.0;
19      }
20
21  private:
22      void do_work()

```

```

23 {
24
25     if (progress_ < 1.0) {
26         progress_ += 0.02;
27         send_feedback(progress_, "Move running");
28     } else {
29         finish(true, 1.0, "Move completed");
30
31         progress_ = 0.0;
32         std::cout << std::endl;
33     }
34
35     std::cout << "\r\e[K" << std::flush;
36     std::cout << "Moving ... [" << std::min(100.0, progress_
37         * 100.0) << "%]  " <<
38         std::flush;
39 }
40
41 float progress_;
42 };
43
44 int main(int argc, char ** argv)
45 {
46     rclcpp::init(argc, argv);
47     auto node = std::make_shared<MoveAction>();
48
49     node->set_parameter(rclcpp::Parameter("action_name", "move
50         "));
51     node->trigger_transition(lifecycle_msgs::msg::Transition::
52         TRANSITION_CONFIGURE);
53
54     rclcpp::spin(node->get_node_base_interface());
55
56     rclcpp::shutdown();
57
58     return 0;
59 }

```

Infine, abbiamo definito opportunamente i file CMakeLists.txt e package.xml per gestire le dipendenze e il processo di build del pacchetto ROS.

File CMakeList.txt:

```
1   cmake_minimum_required(VERSION 3.5)
2   project(robotic_planning_ws)
3
4   find_package(ament_cmake REQUIRED)
5   find_package(rclcpp REQUIRED)
6   find_package(rclcpp_action REQUIRED)
7   find_package(plansys2_msgs REQUIRED)
8   find_package(plansys2_executor REQUIRED)
9
10  set(CMAKE_CXX_STANDARD 17)
11
12  set(dependencies
13      rclcpp
14      rclcpp_action
15      plansys2_msgs
16      plansys2_executor
17  )
18
19  add_executable(move src/move.cpp)
20  ament_target_dependencies(move ${dependencies})
21
22  add_executable(fill src/fill.cpp)
23  ament_target_dependencies(fill ${dependencies})
24
25  add_executable(load src/load.cpp)
26  ament_target_dependencies(load ${dependencies})
27
28  add_executable(unload src/unload.cpp)
29  ament_target_dependencies(unload ${dependencies})
30
31  add_executable(empty src/empty.cpp)
32  ament_target_dependencies(empty ${dependencies})
```

```

33
34 add_executable(emptyatleastone src/emptyatleastone.cpp)
35 ament_target_dependencies(emptyatleastone ${dependencies})
36
37 install(DIRECTORY launch pddl DESTINATION share/${
    PROJECT_NAME})
38
39 install(TARGETS
40     move
41     fill
42     load
43     unload
44     empty
45     emptyatleastone
46     ARCHIVE DESTINATION lib
47     LIBRARY DESTINATION lib
48     RUNTIME DESTINATION lib/${PROJECT_NAME}
49 )
50
51 if(BUILD_TESTING)
52     find_package(ament_lint_auto REQUIRED)
53     ament_lint_auto_find_test_dependencies()
54
55     find_package(ament_cmake_gtest REQUIRED)
56 endif()
57
58 ament_export_dependencies(${dependencies})
59
60 ament_package()

```

File package.xml:

```

1     <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/
    package_format3.xsd" schematypens="http://www.w3.org
    /2001/XMLSchema"?>
3 <package format="3">
4     <name>robotic_planning_ws</name>

```

```

5 <version>0.0.4</version>
6
7 <description>Robotic planning for an emergency scenario</
  description>
8
9 <maintainer email="cim@gmail.com">CIM</maintainer>
10 <license>Apache License, Version 2.0</license>
11 <buildtool_depend>ament_cmake</buildtool_depend>
12
13 <depend>rclcpp</depend>
14 <depend>rclcpp_action</depend>
15 <depend>plansys2_msgs</depend>
16 <depend>plansys2_executor</depend>
17
18 <exec_depend>plansys2_bringup</exec_depend>
19 <exec_depend>plansys2_terminal</exec_depend>
20
21 <test_depend>ament_lint_common</test_depend>
22 <test_depend>ament_lint_auto</test_depend>
23 <test_depend>ament_cmake_gtest</test_depend>
24
25 <export>
26   <build_type>ament_cmake</build_type>
27 </export>
28 </package>

```

3.2.2 Simulazione

Dopo aver definito tutti i file necessari abbiamo eseguito i comandi necessari per avviare la simulazione sul piano ottenuto al punto 3.1.

Innanzitutto è necessario aprire un terminale all'interno del workspace `robotic_planning_ws` ed eseguire i seguenti comandi per compilare il workspace:

```
1 colcon build --symlink-install
2
3 source install/setup.bash
```

Si va poi a avviare i nodi ROS utilizzando il file di lancio `launch.py` con il seguente comando:

```
1 ros2 launch robotic_planning_ws launch.py
```

Successivamente è necessario aprire un secondo terminale ed eseguire il seguente comando per avviare il terminale interattivo di PlanSys2:

```
1 ros2 run plansys2_terminal plansys2_terminal
```

Sempre su questo secondo terminale si carica l'istanza descritta nel file `commands.txt` con il comando:

```
1 source /home/aiguy/plansys2_ws/src/
  ros2_planning_system_examples/progettoAI/
  robotic_planning_ws/launch/commands.txt
```

Infine si esegue il piano tramite il comando:

```
1 run plan-file /home/aiguy/plansys2_ws/src/
  ros2_planning_system_examples/progettoAI/
  robotic_planning_ws/launch/plan.txt
```

La simulazione si completa con successo. Nel primo terminale viene mostrato l'avanzamento dell'esecuzione. L'output ottenuto è il seguente:

```
aiguy@ubu22: ~/plansys2_ws/src/ros2_planning_syst...
Filling ... [100%]
Loading ... [100%]
Filling ... [100%]
Loading ... [100%]
Filling ... [100%]
Filling ... [100%]
Loading ... [100%]
Loading ... [100%]
Moving ... [100%]
Unloading ... [100%]
Emptying ... [100%]
Unloading ... [100%]
Emptying ... [100%]
Unloading ... [100%]
Emptying ... [100%]
Moving ... [100%]
Unloading ... [100%]
Emptying ... [100%]
[plansys2_node-1] [INFO] [1719828134.377901063] [executor]: Plan Succeeded
```

Figura 1: Output nel primo terminale

```
aiguy@ubu22: ~/plansys2_ws/src/ros2_plannin...
> run plan-file /home/aiguy/plansys2_ws/src/ros2_planning_system_exam
ples/progettoAI/robotic_planning_ws/launch/plan.txt
The plan read from "/home/aiguy/plansys2_ws/src/ros2_planning_system_
examples/progettoAI/robotic_planning_ws/launch/plan.txt" is
0.0003: (fill a b2 drug depot) [1]
1.0005: (load a c place1 b2 depot) [4]
5.0008: (fill a b5 drug depot) [1]
6.001: (load a c place2 b5 depot) [4]
10.0013: (fill a b4 food depot) [2]
12.0015: (fill a b3 food depot) [2]
14.0017: (load a c place3 b4 depot) [6]
20.002: (load a c place4 b3 depot) [6]
26.0023: (move a c depot l1) [65]
91.0025: (unload a c place1 b2 l1) [4]
95.0027: (empty a b2 drug p1 l1) [3]
98.003: (unload a c place2 b5 l1) [4]
102.003: (empty a b5 drug p2 l1) [3]
105.004: (unload a c place4 b3 l1) [6]
111.004: (empty a b3 food p1 l1) [6]
117.004: (move a c l1 l2) [30]
147.004: (unload a c place3 b4 l2) [6]
153.005: (empty a b4 food p3 l2) [6]
[INFO] [1719828136.640983172] [executor_client]: Plan Succeeded

Successful finished
>
```

Figura 2: Output nel secondo terminale.